МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №8

з дисципліни «Професійний Пайтон»

на тему: «Мережеве та асинхронне програмування. REST API»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Ознайомитися з основами мережевого програмування в Python, протоколом HTTP, архітектурним стилем REST і форматом JSON. Навчитися створювати REST API на FastAPI з Pydantic-схемами, використовувати async/await і asyncio (Task, gather, timeout, retry, Semaphore), працювати з HTTPX і автоматично тестувати API.

2. Індивідуальне завдання

Варіант 15. Облік результатів наукових експериментів. Ресурси: researchers, experiments. Endpoints: `GET /experiments`, `GET /experiments/{id}`, `POST /experiments`, `PATCH /experiments/{id}`, `DELETE /experiments/{id}`, `GET /researchers/{id}/experiments`, `GET /experiments/statistics`. Реалізувати: додавання researcher та experiment; пошук за researcher; зміну результату; видалення; average, minimum, maximum; фільтрацію результатів за діапазоном; сортування за датою. Асинхронне завдання: паралельно отримувати статистику для декількох researchers; Task/gather, Semaphore, HTTPX, timeout, retry.

3. Постановка задачі

Проєкт попередніх лабораторних потрібно розширити REST API, що працює з persistence layer лабораторної №7: щонайменше 5 endpoints з методами GET, POST, PATCH, DELETE, JSON і Pydantic-валідацією, path і query parameters, коректні HTTP status codes; асинхронні функції з `await`, `asyncio.Task`, `asyncio.gather`, timeout, retry, `asyncio.Semaphore`; HTTP-запити через `httpx.AsyncClient`; автоматизовані тести API.

4. Хід виконання

4.1. Архітектура та структура проєкту

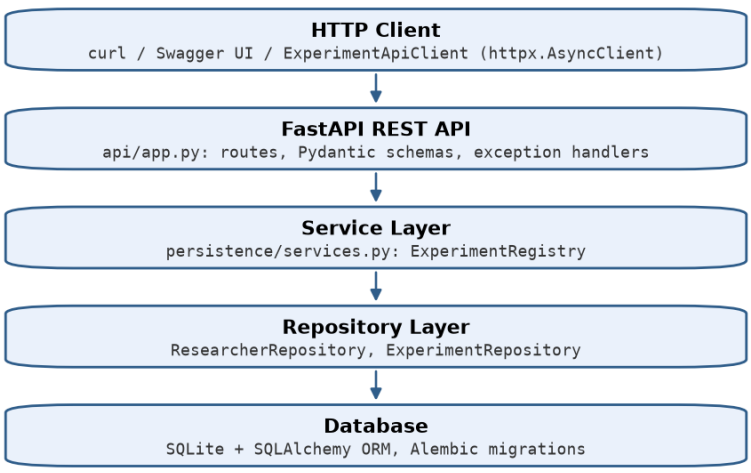


Рис. 8.1. Шари застосунку

Лістинг 8.1. Нові модулі: пакет api і тести API

```
api/
├── __init__.py
├── aggregation.py
├── app.py
├── external.py
└── schemas.py

api/
├── conftest.py
├── test_api.py
└── test_async_client.py
```

Таблиця 8.1. Нові та змінені модулі

Модуль	Призначення
`api/schemas.py`	Pydantic-моделі запитів і відповідей, правила валідації
`api/app.py`	фабрика `create_app()`, endpoints, обробники винятків, lifespan
`api/aggregation.py`	паралельна статистика: Task + gather + Semaphore + timeout
`api/external.py`	`ExperimentApiClient`: httpx.AsyncClient, retry з backoff, Semaphore
`commands/api_client.py`	підкоманди `serve` (uvicorn) і `api-client`
`persistence/services.py`	нові методи сервісу для API (фільтри, сортування, PATCH)

4.2. Опис REST API та список endpoints

API побудовано за принципами REST: ресурси адресуються іменниками (`/researchers`, `/experiments`), дія визначається HTTP-методом, дані передаються в JSON, сервер не зберігає стану клієнта між запитами. Повна документація OpenAPI генерується автоматично і доступна в Swagger UI (`/docs`).

Таблиця 8.2. Endpoints API

Method	Endpoint	Опис	Status codes
GET	`/researchers`	список дослідників	200
POST	`/researchers`	додати дослідника	201, 409, 422
GET	`/researchers/{researcher_id}`	дослідник за ID	200, 404
GET	`/researchers/{researcher_id}/experiments`	експерименти дослідника	200, 404
GET	`/experiments`	список з фільтрами, сортуванням, пагінацією	200, 422
POST	`/experiments`	додати експеримент	201, 404, 422
GET	`/experiments/{experiment_id}`	експеримент за ID	200, 404, 422
PATCH	`/experiments/{experiment_id}`	часткова зміна (результат тощо)	200, 404, 422
DELETE	`/experiments/{experiment_id}`	видалення	204, 404
GET	`/experiments/statistics`	average, minimum, maximum	200, 404
GET	`/experiments/statistics/by-researcher`	паралельна статистика кількох дослідників	200, 422

Таблиця 8.3. Таблиця HTTP methods

HTTP method	Семантика	Використання в API
GET	читання, безпечний та ідемпотентний	списки, об’єкт, статистика
POST	створення нового ресурсу, не ідемпотентний	`/researchers`, `/experiments` → 201 Created

HTTP method	Семантика	Використання в API
PATCH	часткова зміна ресурсу	зміна результату, дати, примітки
DELETE	видалення, ідемпотентний за станом	`/experiments/{id}` → 204 No Content

4.3. Path і query parameters

Path parameters ідентифікують ресурс: `/experiments/{experiment_id}`, `/researchers/{researcher_id}/experiments`; FastAPI перетворює їх на `int` і повертає 422 для `/experiments/abc`. Query parameters задають фільтри та налаштування: `researcher_id`, `parameter`, `min_result`, `max_result`, `sort` (`date`, `-date`, `result`, `-result`), `limit` (1–100), `offset`; для паралельної статистики — список `ids`, `max_concurrency`, `timeout`. Маршрут `/experiments/statistics` оголошено раніше за `/experiments/{experiment_id}`, інакше слово `statistics` намагалися б перетворити на ID.

Лістинг 8.2. Приклади path і query parameters

```
GET /experiments/3
GET /researchers/2/experiments?sort=date
GET /experiments?researcher_id=1&sort=-result&limit=2
GET /experiments?parameter=temperature&min_result=100&max_result=300
GET /experiments/statistics?parameter=temperature
GET /experiments/statistics/by-researcher?ids=1&ids=2&ids=3&max_concurrency=2
```

4.4. Pydantic schemas і validation rules

Лістинг 8.3. Pydantic-схеми (фрагмент)

```
"""Pydantic v2 schemas: request bodies, responses and validation rules."""

import datetime as dt
from typing import Annotated, Literal

from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator

Parameter = Annotated[str, Field(min_length=1, max_length=50, pattern=r"^[a-z_]+$",
                                 examples=["temperature"])]
Result = Annotated[float, Field(ge=0, le=1_000_000, allow_inf_nan=False, examples=[293.4])]
SortOrder = Literal["date", "-date", "result", "-result"]


def _not_in_future(value: dt.date | None) -> dt.date | None:
    if value is not None and value > dt.date.today():
        raise ValueError("date cannot be in the future")
    return value


class ResearcherCreate(BaseModel):
    full_name: str = Field(min_length=3, max_length=120, examples=["Olena Kovalenko"])
    email: str | None = Field(default=None, pattern=r"^[^@\s]+@[^@\s]+\.[^@\s]+$",
                              examples=["o.kovalenko@lnu.edu.ua"])
    department: str = Field(default="Physics", max_length=80)


class ResearcherRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    full_name: str
    email: str | None
    department: str


class ExperimentCreate(BaseModel):
    name: str = Field(min_length=1, max_length=200, examples=["Thermal conductivity of Al alloy"])
    researcher_id: int = Field(gt=0, examples=[1])
    parameter: Parameter
```

```
    result: Result
    date: dt.date
    notes: str | None = Field(default=None, max_length=500)

    _check_date = field_validator("date")(_not_in_future)


class ExperimentUpdate(BaseModel):
    """PATCH body: every field is optional, only the sent fields are changed."""

    model_config = ConfigDict(extra="forbid")

    name: str | None = Field(default=None, min_length=1, max_length=200)
    parameter: Parameter | None = None
    result: Result | None = None
    date: dt.date | None = None  # module alias: a field named `date` would shadow the type
    notes: str | None = Field(default=None, max_length=500)

    _check_date = field_validator("date")(_not_in_future)

    @model_validator(mode="after")
    def at_least_one_field(self) -> "ExperimentUpdate":
        if not self.model_fields_set:
            raise ValueError("at least one field must be provided")
        return self


class ExperimentRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    researcher_id: int
    researcher: str
    parameter: str
    result: float
    date: dt.date
    notes: str | None

    @field_validator("researcher", mode="before")
    @classmethod
    def researcher_name(cls, value: object) -> object:
        return getattr(value, "full_name", value)


class StatisticsRead(BaseModel):
    count: int
```

Таблиця 8.4. Validation rules

Поле	Правило	Помилка
`name`	1–200 символів	422 string_too_short
`researcher_id`	ціле > 0; дослідник має існувати	422 / 404
`parameter`	1–50 символів, шаблон `^[a-z_]+$`	422 string_pattern_mismatch
`result`	число 0 … 1 000 000, без NaN/Inf	422 greater_than_equal
`date`	ISO-дата, не в майбутньому	422 value_error
`notes`	до 500 символів, необов’язкове	422
PATCH body	хоча б одне поле; невідомі поля заборонені (`extra=forbid`)	422
`email`	шаблон адреси, унікальне	422 / 409

4.5. Код FastAPI application

Застосунок створюється фабрикою `create_app(database_url)`: це дозволяє тестам підключати окрему базу. Сервіс зберігається в `app.state` і передається в endpoints через dependency injection (`Depends(get_registry)`). Власні винятки доменного шару перетворюються на HTTP-відповіді обробниками:

`ExperimentNotFoundError` → 404, `ExperimentValidationError` → 422, `DuplicateExperimentError` → 409. Об'єкт `experiment_registry.api:app` для uvicorn створюється ліниво (через `__getattr__` модуля), щоб імпорт схем чи клієнта не відкривав базу.

Лістинг 8.4. Dependency, фабрика застосунку та обробники винятків

```
def get_registry(request: Request) -> ExperimentRegistry:
    """Dependency: the service object stored in application state."""
    registry: ExperimentRegistry = request.app.state.registry
    return registry


Registry = Annotated[ExperimentRegistry, Depends(get_registry)]


def to_read(items: Sequence[ExperimentORM]) -> list[ExperimentRead]:
    return [ExperimentRead.model_validate(item) for item in items]


def create_app(database_url: str | None = None, create_schema: bool = True) -> FastAPI:
    url = database_url or get_database_url()
    engine = create_db_engine(url)
    if create_schema and not inspect(engine).has_table("experiments"):
        init_db(engine)  # development convenience; production uses `alembic upgrade head`

    @asynccontextmanager
    async def lifespan(_: FastAPI) -> AsyncIterator[None]:
        yield
        engine.dispose()

    app = FastAPI(
        title="Experiment Registry API",
        version=__version__,
        description="REST API for scientific experiment results (variant 15, Professional Python).",
        lifespan=lifespan,
    )
    app.state.engine = engine
    app.state.registry = ExperimentRegistry(create_session_factory(engine))

    # ------------------------------------------------------------ error handlers
    @app.exception_handler(ExperimentNotFoundError)
    async def not_found(_: Request, error: ExperimentNotFoundError) -> JSONResponse:
        return JSONResponse(status_code=404, content={"detail": str(error)})

    @app.exception_handler(ExperimentValidationError)
    async def invalid(_: Request, error: ExperimentValidationError) -> JSONResponse:
        return JSONResponse(status_code=422, content={"detail": str(error)})

    @app.exception_handler(DuplicateExperimentError)
    async def conflict(_: Request, error: DuplicateExperimentError) -> JSONResponse:
        return JSONResponse(status_code=409, content={"detail": str(error)})
```

4.6. Реалізація GET, POST, PATCH і DELETE

CRUD-endpoints оголошено звичайними `def`: робота з SQLite синхронна, і FastAPI виконує такі функції в пулі потоків, не блокуючи event loop. Endpoints статистики оголошено `async def`, бо вони чекають (`await`) на виконання в потоці чи на групу задач.

Лістинг 8.5. GET /experiments з query parameters

```
    @app.get("/experiments", response_model=list[ExperimentRead], tags=["experiments"])
    def list_experiments(
        registry: Registry,
        researcher_id: Annotated[int | None, Query(gt=0)] = None,
        parameter: str | None = None,
        min_result: float | None = None,
        max_result: float | None = None,
        sort: SortOrder = "date",
        limit: Annotated[int, Query(ge=1, le=100)] = 50,
        offset: Annotated[int, Query(ge=0)] = 0,
    ) -> list[ExperimentRead]:
```

```
        if min_result is not None and max_result is not None and min_result > max_result:
            raise HTTPException(status_code=422, detail="min_result must be <= max_result")
        return to_read(registry.list_experiments(
            researcher_id=researcher_id, parameter=parameter, min_result=min_result,
            max_result=max_result, sort=sort, limit=limit, offset=offset))
```

Лістинг 8.6. POST, PATCH і DELETE

```
    @app.post("/experiments", response_model=ExperimentRead, status_code=status.HTTP_201_CREATED,
              responses=NOT_FOUND, tags=["experiments"])
    def create_experiment(body: ExperimentCreate, registry: Registry) -> ExperimentRead:
        created = registry.create_experiment(body.researcher_id, body.name, body.parameter,
body.result,
                                             body.date, body.notes)
        return ExperimentRead.model_validate(registry.get_experiment(created.id))

    @app.patch("/experiments/{experiment_id}", response_model=ExperimentRead, responses=NOT_FOUND,
               tags=["experiments"])
    def update_experiment(experiment_id: int, body: ExperimentUpdate, registry: Registry) ->
ExperimentRead:
        registry.update_experiment(experiment_id, body.model_dump(exclude_unset=True))
        return ExperimentRead.model_validate(registry.get_experiment(experiment_id))

    @app.delete("/experiments/{experiment_id}", status_code=status.HTTP_204_NO_CONTENT,
                responses=NOT_FOUND, tags=["experiments"])
    def delete_experiment(experiment_id: int, registry: Registry) -> Response:
        registry.delete(experiment_id)
        return Response(status_code=status.HTTP_204_NO_CONTENT)

    return app
```

4.7. Приклади JSON request і response

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab08$ alembic upgrade head
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade  -> 0001, create researchers and experiments
INFO  [alembic.runtime.migration] Running upgrade 0001 -> 0002, add experiment notes
```

Рис. 8.2. Підготовка бази даних міграціями

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab08$ curl -s -X POST http://127.0.0.1:8000/researchers -H
'Content-Type: application/json' -d '{"full_name": "Olena Kovalenko", "email": "o.kovalenko@lnu.edu.ua"}'
{"id":1,"full_name":"Olena Kovalenko","email":"o.kovalenko@lnu.edu.ua","department":"Physics"}
(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab08$ curl -s -X POST http://127.0.0.1:8000/researchers -H
'Content-Type: application/json' -d '{"full_name": "Taras Hnatiuk", "department": "Mechanics"}'
{"id":2,"full_name":"Taras Hnatiuk","email":null,"department":"Mechanics"}
(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab08$ curl -s -X POST http://127.0.0.1:8000/researchers -H
'Content-Type: application/json' -d '{"full_name": "Iryna Melnyk"}'
{"id":3,"full_name":"Iryna Melnyk","email":null,"department":"Physics"}
```

Рис. 8.3. POST /researchers: створення дослідників (201 Created)

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab08$ curl -s -X POST http://127.0.0.1:8000/experiments -H
'Content-Type: application/json' -d '{"name": "Thermal conductivity of Al alloy", "researcher_id": 1,
"parameter": "temperature", "result": 293.4, "date": "2026-02-03"}' -w '\nHTTP %{http_code}\n'
{"id":1,"name":"Thermal conductivity of Al alloy","researcher_id":1,"researcher":"Olena
Kovalenko","parameter":"temperature","result":293.4,"date":"2026-02-03","notes":null}
HTTP 201
```

Рис. 8.4. POST /experiments: JSON request і JSON response

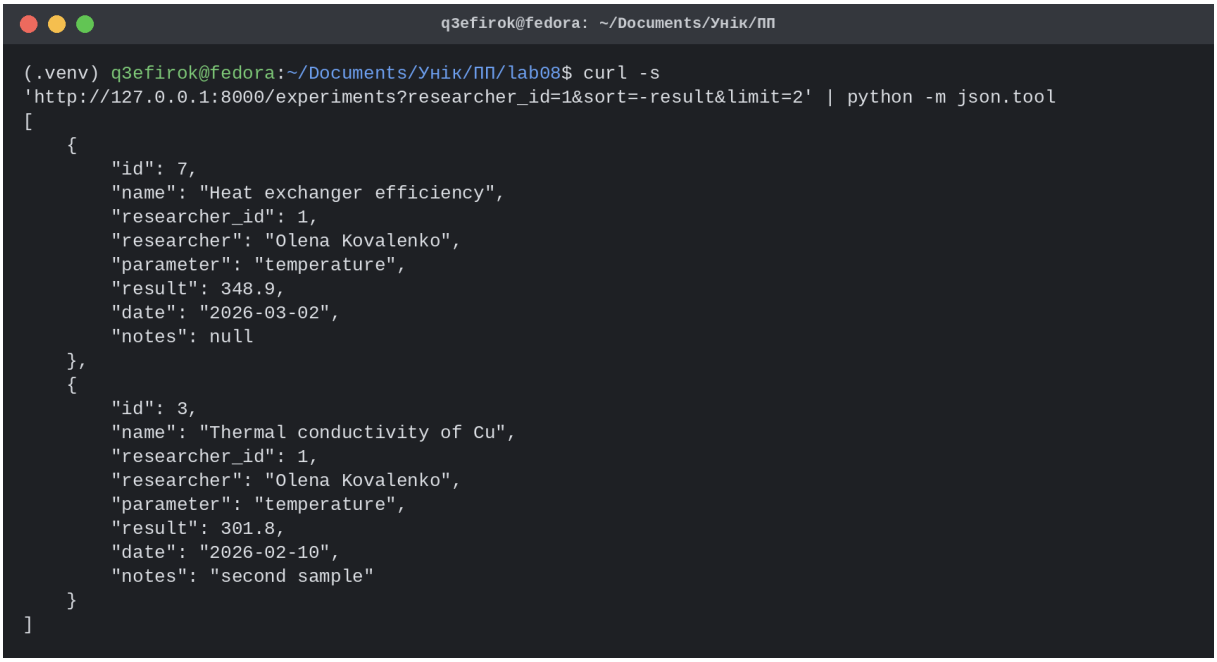


Рис. 8.5. GET з query parameters: фільтр за дослідником, сортування, limit

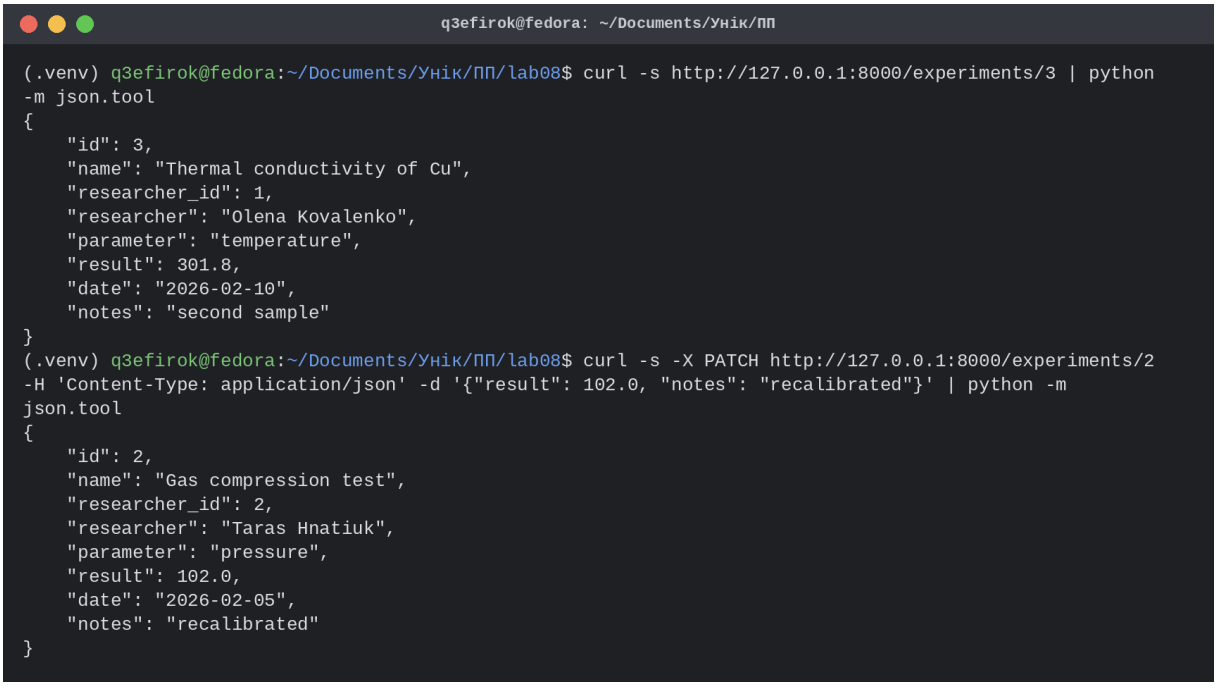


Рис. 8.6. GET за path parameter і PATCH (зміна результату й примітки)



Рис. 8.7. DELETE повертає 204, повторний GET — 404

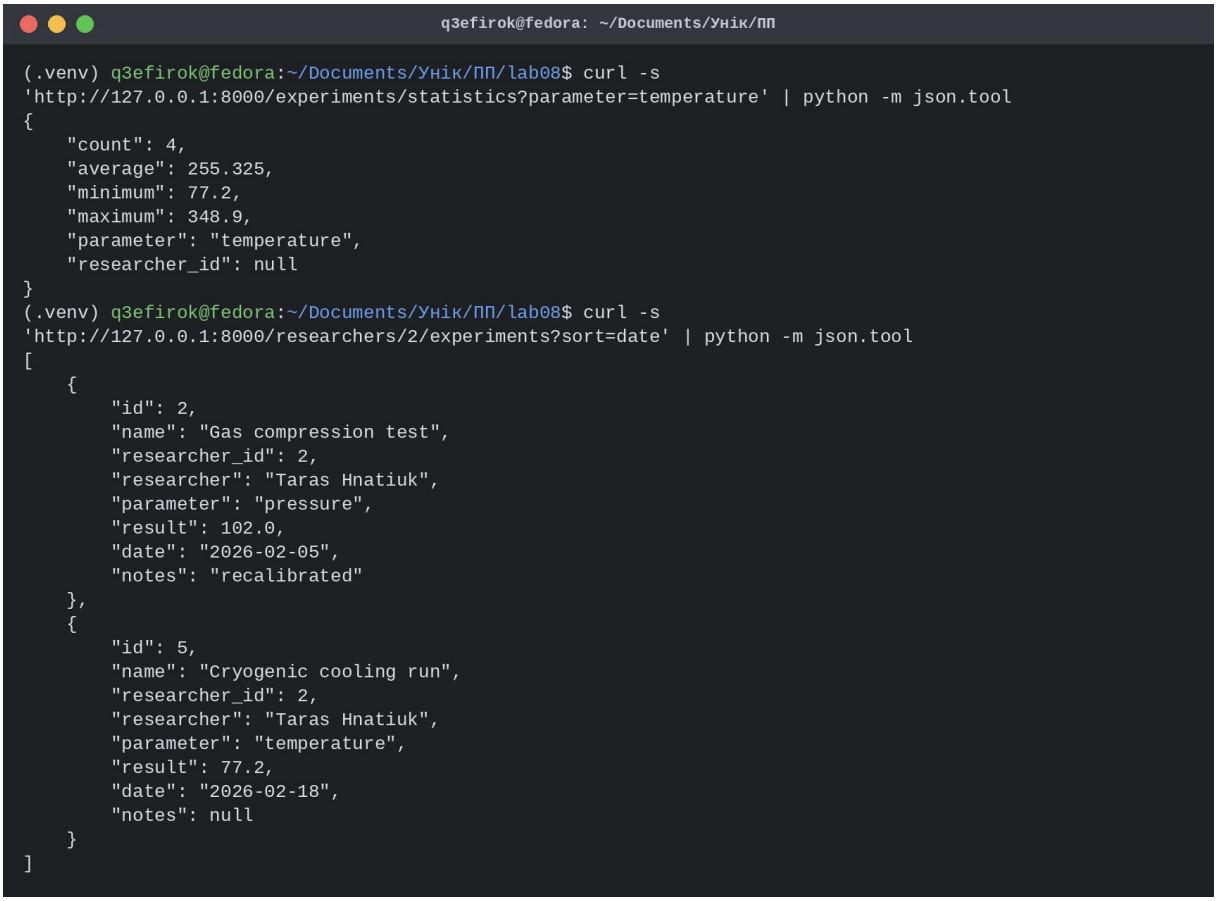


Рис. 8.8. Статистика за параметром і експерименти дослідника



Рис. 8.9. Validation: некоректний JSON отримує 422 з описом кожної помилки

4.8. async/await, Task, gather, Semaphore і timeout

`collect_statistics` для кожного дослідника створює `asyncio.Task`; усередині задачі `async with semaphore` обмежує кількість одночасних звернень до бази, `asyncio.to_thread` переносить блокуючий SQL-запит у потік, а `asyncio.wait_for` обриває занадто повільне обчислення. `asyncio.gather(..., return_exceptions=True)` чекає на всі задачі, тож помилка одного дослідника (наприклад, 404) не скасовує результатів інших.

Лістинг 8.7. Модуль aggregation.py

```
"""Concurrent statistics for several researchers: Task + gather + Semaphore + timeout."""
```

```
import asyncio
import time
from collections.abc import Callable, Sequence

from experiment_registry.api.schemas import AggregatedStatistics, ResearcherStatistics,
StatisticsRead
from experiment_registry.persistence.repositories import ResultSummary


async def collect_statistics(
    load: Callable[[int], ResultSummary],
    researcher_ids: Sequence[int],
    max_concurrency: int = 3,
    timeout: float = 2.0,
) -> AggregatedStatistics:
    """Run blocking `load(researcher_id)` calls in threads, at most `max_concurrency` at
once."""
    semaphore = asyncio.Semaphore(max_concurrency)

    async def one(researcher_id: int) -> ResultSummary:
        async with semaphore:
            return await asyncio.wait_for(asyncio.to_thread(load, researcher_id),
timeout=timeout)

    start = time.perf_counter()
    tasks = [asyncio.create_task(one(researcher_id), name=f"stats-{researcher_id}")
             for researcher_id in researcher_ids]
    outcomes = await asyncio.gather(*tasks, return_exceptions=True)
    items = []
    for researcher_id, outcome in zip(researcher_ids, outcomes, strict=True):
        if isinstance(outcome, BaseException):
            reason = "timeout" if isinstance(outcome, TimeoutError) else str(outcome)
            items.append(ResearcherStatistics(researcher_id=researcher_id, error=reason))
        else:
            items.append(ResearcherStatistics(researcher_id=researcher_id,
statistics=StatisticsRead(
                count=outcome.count, average=outcome.average, minimum=outcome.minimum,
                maximum=outcome.maximum, researcher_id=researcher_id)))
    return AggregatedStatistics(items=items, elapsed_ms=round((time.perf_counter() - start) *
1000, 2),
                                max_concurrency=max_concurrency)
```

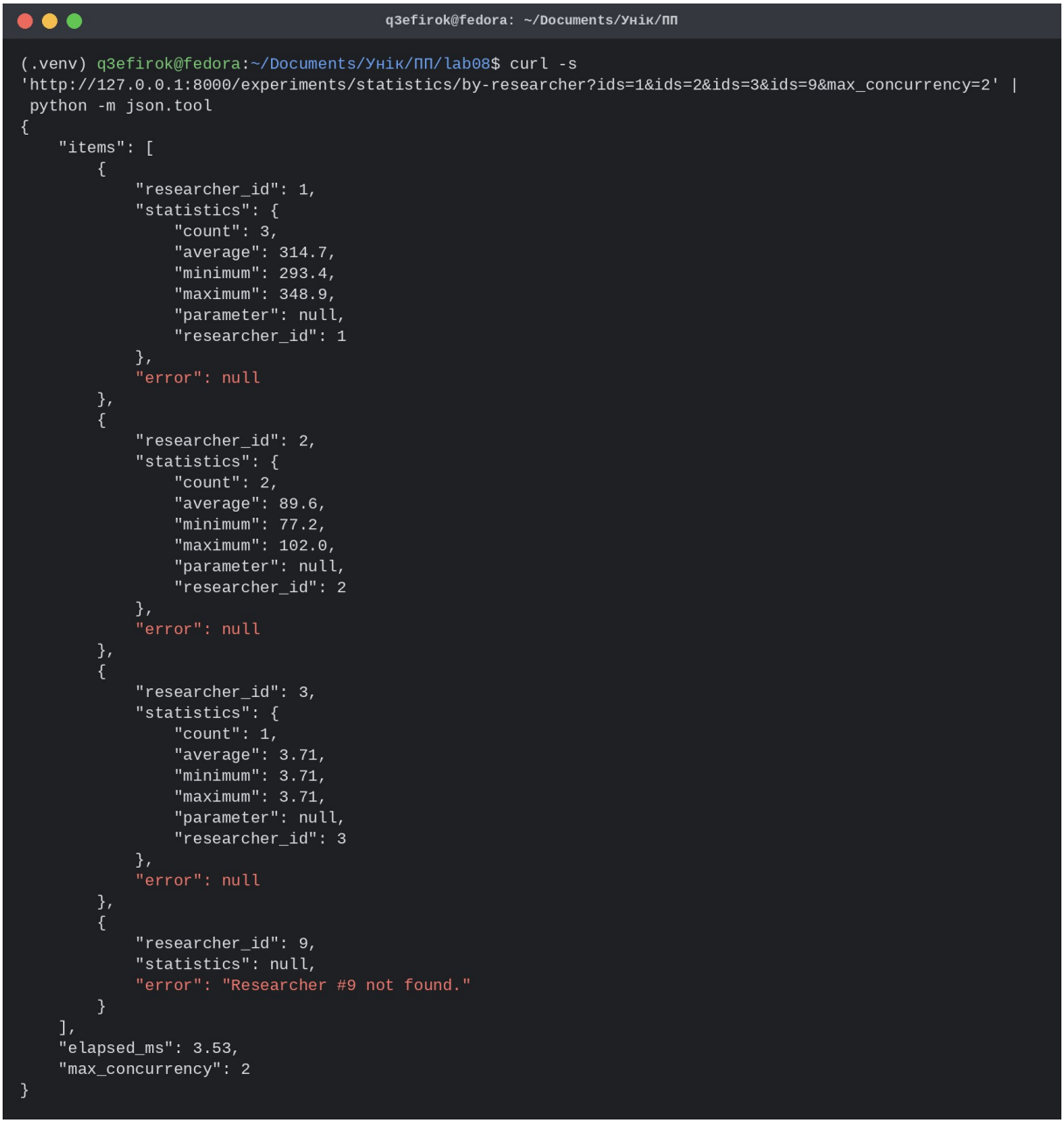


Рис. 8.10. Паралельна статистика для чотирьох дослідників (ID 9 не існує)

4.9. HTTPX і retry

`ExperimentApiClient` — асинхронний клієнт до власного API, що імітує зовнішній HTTP-сервіс. Усі запити йдуть через один `httpx.AsyncClient` (спільний пул з'єднань), відкритий у `async with`; timeout задається `httpx.Timeout`. `get_json` повторює запит при мережевих помилках, timeout і статусах 502–504 з експоненційною затримкою (0.2, 0.4, 0.8 с), а помилки 4xx не повторює, бо повторний запит їх не виправить. `fetch_many` запускає запити для всіх дослідників через Task і gather під `Semaphore`.

Лістинг 8.8. Асинхронний клієнт HTTPX з retry

```
@dataclass
class ExperimentApiClient:
    """Calls GET /experiments/statistics for many researchers concurrently.

    One AsyncClient (connection pool) is shared by all requests inside `async with`.
```

```
    """

    base_url: str
    timeout: float = 2.0
    retries: int = 3
    backoff: float = 0.2
    max_concurrency: int = 4
    transport: httpx.AsyncBaseTransport | None = None
    _client: httpx.AsyncClient | None = field(default=None, init=False, repr=False)

    async def __aenter__(self) -> "ExperimentApiClient":
        self._client = httpx.AsyncClient(base_url=self.base_url, timeout=httpx.Timeout(self.timeout),
                                         transport=self.transport)
        return self

    async def __aexit__(self, *exc_info: object) -> None:
        if self._client is not None:
            await self._client.aclose()

    async def get_json(self, path: str, **params: Any) -> tuple[Any, int]:
        """GET with retry and exponential backoff for network errors, timeouts and 502-504."""
        assert self._client is not None, "use 'async with ExperimentApiClient(...)'"
        for attempt in range(1, self.retries + 1):
            try:
                response = await self._client.get(path, params=params)
                if response.status_code in RETRY_STATUSES:
                    raise httpx.HTTPStatusError(f"server error {response.status_code}",
                                                request=response.request, response=response)
                response.raise_for_status()
                return response.json(), attempt
            except (httpx.TransportError, httpx.HTTPStatusError) as error:
                retryable = not isinstance(error, httpx.HTTPStatusError) or (
                    error.response.status_code in RETRY_STATUSES)
                if not retryable or attempt == self.retries:
                    raise
                delay = self.backoff * 2 ** (attempt - 1)
                logger.warning("attempt %d for %s failed (%s), retry in %.2f s", attempt, path,
                               type(error).__name__, delay)
                await asyncio.sleep(delay)
        raise RuntimeError("unreachable")

    async def fetch_statistics(self, researcher_id: int, semaphore: asyncio.Semaphore) -> FetchResult:
        result = FetchResult(researcher_id)
        start = time.perf_counter()
        async with semaphore:
            try:
                result.data, result.attempts = await self.get_json(
                    "/experiments/statistics", researcher_id=researcher_id)
            except httpx.HTTPStatusError as error:
                result.error = f"HTTP {error.response.status_code}"
                result.attempts = self.retries if error.response.status_code in RETRY_STATUSES else 1
            except httpx.TimeoutException:
                result.error, result.attempts = "timeout", self.retries
            except httpx.TransportError as error:
                result.error, result.attempts = type(error).__name__, self.retries
        result.elapsed_ms = round((time.perf_counter() - start) * 1000, 1)
        return result

    async def fetch_many(self, researcher_ids: Sequence[int]) -> list[FetchResult]:
        semaphore = asyncio.Semaphore(self.max_concurrency)
        tasks = [asyncio.create_task(self.fetch_statistics(rid, semaphore)) for rid in researcher_ids]
        return list(await asyncio.gather(*tasks))
```

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab08$ experiment-registry api-client --base-url
http://127.0.0.1:8000 --researchers 1 2 3 9 --concurrency 2
GET http://127.0.0.1:8000/experiments/statistics?researcher_id=... (timeout=2.0s, retries=3,
Semaphore(2))
  researcher 1: count=3 average=314.7 min=293.4 max=348.9  [1 attempt(s), 19.5 ms]
  researcher 2: count=2 average=89.6 min=77.2 max=102.0  [1 attempt(s), 5.7 ms]
  researcher 3: count=1 average=3.71 min=3.71 max=3.71  [1 attempt(s), 7.6 ms]
  researcher 9: ERROR HTTP 404  [1 attempt(s), 7.9 ms]
Total: 4 requests in 42.2 ms
```

Рис. 8.11. Клієнт паралельно запитує статистику в запущеного сервера

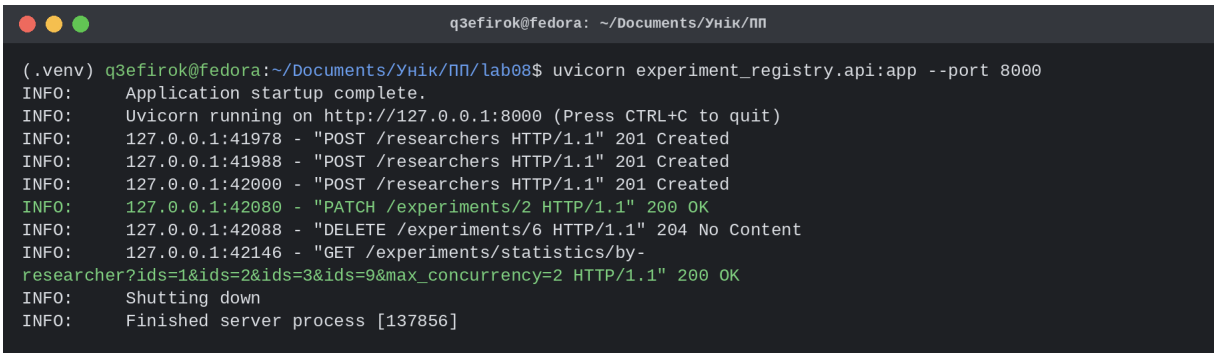


Рис. 8.12. Журнал сервера uvicorn під час демонстрації

4.10. Swagger UI

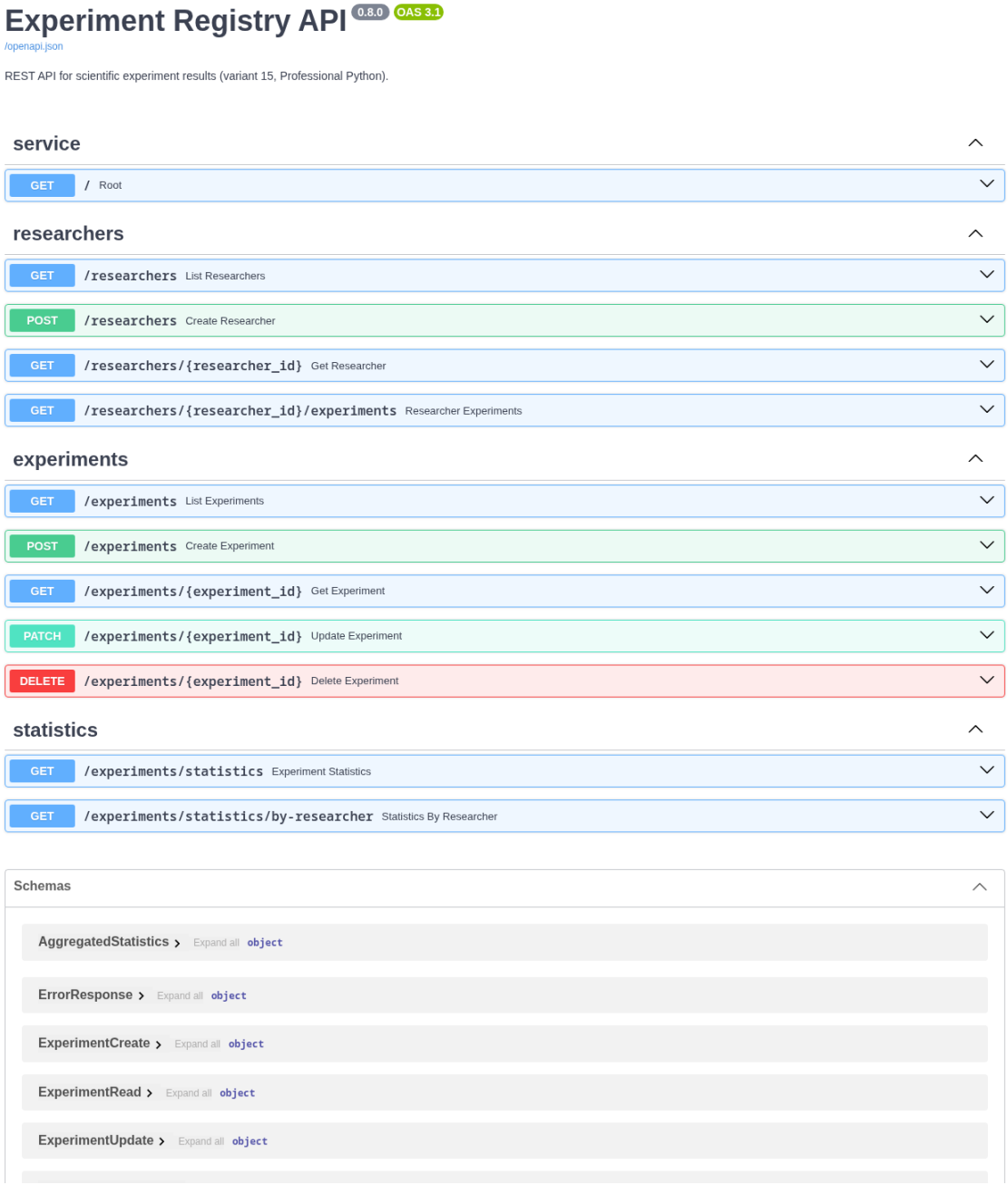


Рис. 8.13. Swagger UI (http://127.0.0.1:8000/docs) з усіма endpoints і схемами

4.11. API tests

Синхронні тести використовують `fastapi.testclient.TestClient` (поточна версія Starlette радить для нього транспорт `httpx2`, доданий до dev-залежностей), асинхронні — `httpx.AsyncClient` з `ASGITransport`, який викликає застосунок напряму без мережі. Retry, timeout і Semaphore перевіряються через `httpx.MockTransport`, що імітує збої сервера. Кожен тест має власну SQLite-базу в `tmp_path`.

Лістинг 8.9. Fixtures API-тестів

```
@pytest.fixture
def api_app(tmp_path: Path) -> Iterator[FastAPI]:
    """Application bound to a separate SQLite database for every test."""
    app = create_app(f"sqlite:///{tmp_path / 'api_test.db'}")
    yield app
    app.state.engine.dispose()  # ASGITransport does not run the lifespan handler


@pytest.fixture
def client(api_app: FastAPI) -> Iterator[TestClient]:
    with TestClient(api_app) as test_client:  # runs lifespan (engine.dispose on exit)
        yield test_client


@pytest.fixture
def seeded(client: TestClient) -> TestClient:
    for name, email in (("Olena Kovalenko", "o.kovalenko@lnu.edu.ua"), ("Taras Hnatiuk", None)):
        assert client.post("/researchers", json={"full_name": name, "email": email}).status_code
== 201
    for body in EXPERIMENTS:
        assert client.post("/experiments", json=body).status_code == 201
    return client
```

Лістинг 8.10. Приклади тестів REST API

```
def test_patch_changes_only_sent_fields(seeded: TestClient) -> None:
    before = seeded.get("/experiments/1").json()

    response = seeded.patch("/experiments/1", json={"result": 295.0})

    assert response.status_code == 200
    assert response.json() == before | {"result": 295.0}


@pytest.mark.parametrize(
    ("change", "field"),
    [({"result": -1}, "result"), ({"result": "abc"}, "result"), ({"parameter": "Temp!"},
"parameter"),
     ({"name": ""}, "name"), ({"date": "2031-01-01"}, "date"), ({"date": "03.02.2026"}, "date"),
     ({"researcher_id": 0}, "researcher_id")],
    ids=["negative", "not-number", "bad-parameter", "empty-name", "future-date", "bad-date",
"bad-id"],
)
def test_invalid_experiment_body_returns_422(seeded: TestClient, change: dict[str, Any], field:
str) -> None:
    body = {"name": "Run", "researcher_id": 1, "parameter": "temperature", "result": 1.0,
            "date": "2026-02-03"} | change

    response = seeded.post("/experiments", json=body)

    assert response.status_code == 422
    assert response.json()["detail"][0]["loc"] == ["body", field]
```

Лістинг 8.11. Тести retry і Semaphore

```
@pytest.mark.asyncio
async def test_retry_with_backoff_after_server_errors() -> None:
    calls = 0
```

```
    def handler(request: httpx.Request) -> httpx.Response:
        nonlocal calls
        calls += 1
        if calls < 3:
            return httpx.Response(503)
        return httpx.Response(200, json={"count": 1, "average": 1.0, "minimum": 1.0, "maximum":
1.0})

    async with ExperimentApiClient("http://test", transport=httpx.MockTransport(handler),
backoff=0.01) as client:
        [result] = await client.fetch_many([1])

    assert result.data is not None and result.attempts == 3 and calls == 3


@pytest.mark.asyncio
async def test_semaphore_limits_parallel_http_requests() -> None:
    active = peak = 0

    async def handler(request: httpx.Request) -> httpx.Response:
        nonlocal active, peak
        active += 1
        peak = max(peak, active)
        await asyncio.sleep(0.02)
        active -= 1
        return httpx.Response(200, json={"count": 0, "average": None, "minimum": None,
"maximum": None})

    async with ExperimentApiClient("http://test", transport=httpx.MockTransport(handler),
                                   max_concurrency=2) as client:
        results = await client.fetch_many(list(range(1, 7)))

    assert peak == 2 and all(item.error is None for item in results)
```

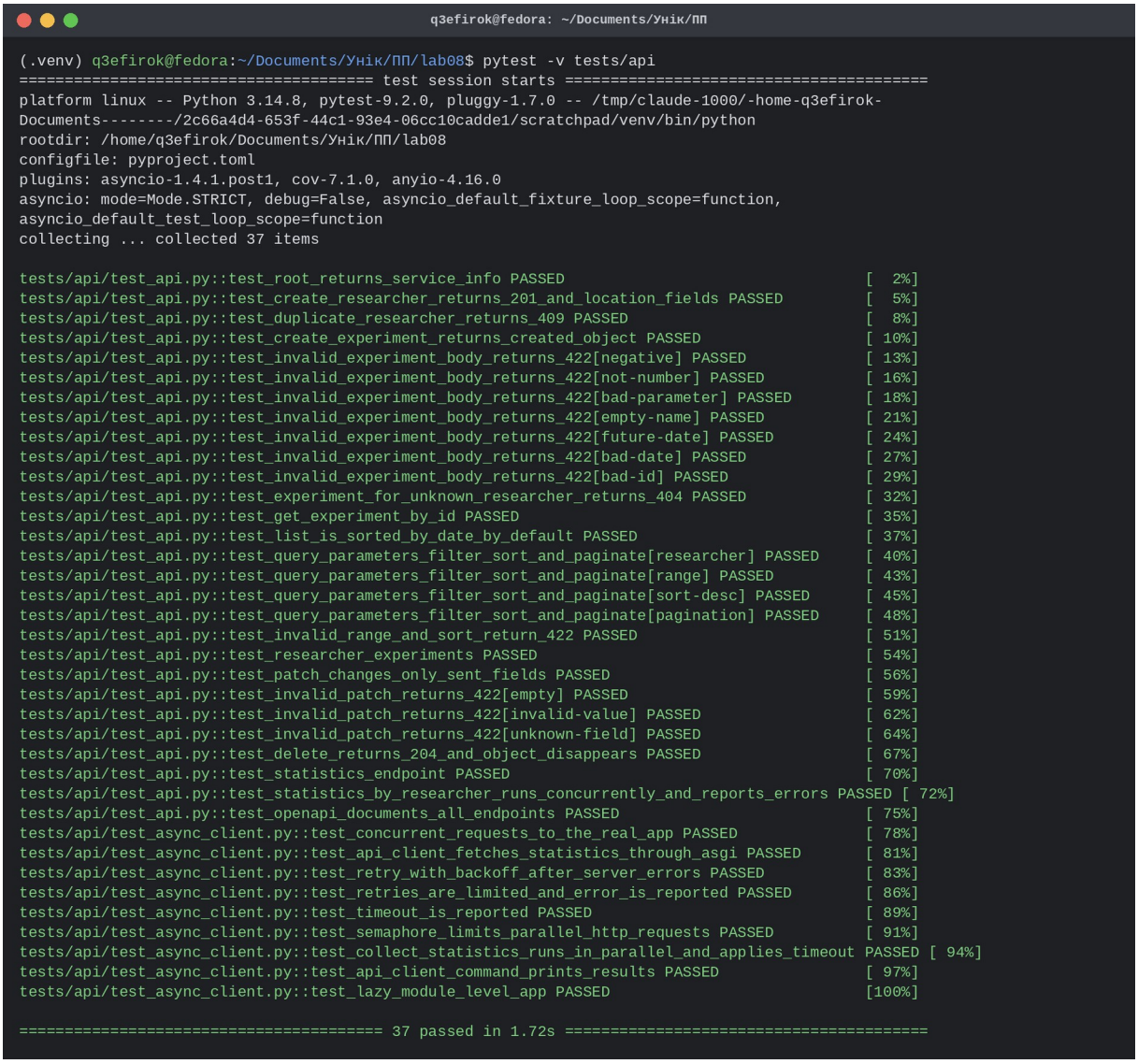


Рис. 8.14. Результати pytest -v для API

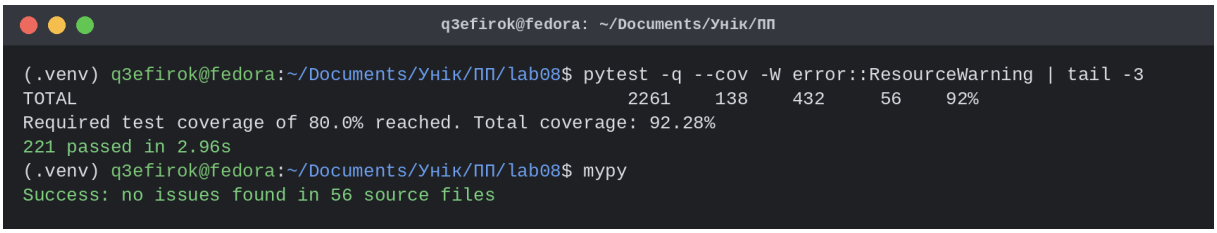


Рис. 8.15. Увесь test suite з coverage і перевірка mypy

Під час тестування виявлено дві проблеми, які виправлено: поле Pydantic-моделі з назвою `date` затіняло тип `date` у тілі класу (рішення — `import datetime as dt`), а в асинхронних тестах з `ASGITransport` lifespan не виконується, тому engine лишав незакрите з'єднання (рішення — `app.state.engine.dispose()` у fixture).

4.12. Завдання підвищеної складності

- pagination, filtering, sorting і кілька query parameters;
- custom exception handlers і структуровані відповіді про помилки;

- dependency injection (`Depends`) і фабрика застосунку;
- паралельні HTTP requests, exponential backoff, configurable timeout і retry count;
- обмеження паралельності через Semaphore;
- OpenAPI metadata, API tests, mock зовнішнього HTTP service (`MockTransport`);
- lifecycle management для HTTP client (`async with`) і engine (`lifespan`).

5. Відповіді на контрольні запитання

1. Що таке комп'ютерна мережа?

Комп'ютерна мережа — сукупність пристроїв (комп'ютерів, серверів, маршрутизаторів), з'єднаних каналами зв'язку, які обмінюються даними за спільними протоколами, наприклад стеком TCP/IP. Мережеве програмування — це створення програм, що взаємодіють через мережу, як-от REST API проєкту та HTTP-клієнт, який до нього звертається.

2. Що таке клієнт-серверна архітектура?

Клієнт-серверна архітектура — модель, у якій клієнт ініціює запит, а сервер обробляє його, звертаючись до своїх ресурсів (наприклад, БД), і повертає відповідь. Ролі чітко розділені, а взаємодія йде за протоколом, наприклад HTTP; у проєкті сервером є FastAPI-застосунок experiment_registry.api:app, а клієнтами — браузер, Swagger UI чи httpx.AsyncClient.

3. Що таке HTTP?

HTTP (HyperText Transfer Protocol) — протокол прикладного рівня для передавання даних між клієнтом і сервером за моделлю запит-відповідь, зазвичай поверх TCP. Він не зберігає стану між запитами (stateless) і визначає методи (GET, POST, PUT, PATCH, DELETE), заголовки та коди стану; HTTPS — це HTTP, захищений TLS.

4. Що містить HTTP request?

HTTP request містить стартовий рядок із методом, шляхом (URL, можливо з query-параметрами) і версією протоколу, заголовки (Host, Content-Type, Accept, Authorization тощо) та необов'язкове тіло. Наприклад, POST /experiments HTTP/1.1 із заголовком Content-Type: application/json і JSON-тілом з даними нового експерименту.

5. Що містить HTTP response?

HTTP response містить рядок стану з версією протоколу, кодом стану й текстовою фразою (наприклад, HTTP/1.1 200 OK), заголовки (Content-Type, Content-Length тощо) та необов'язкове тіло з даними. У проєкті тілом зазвичай є JSON з даними експерименту чи дослідника, а відповідь 204 на DELETE тіла не містить.

6. Для чого використовується GET?

GET використовується для отримання (читання) ресурсу без зміни стану сервера: він безпечний, ідемпотентний, не має тіла запиту, а параметри передаються в шляху або query string. У проєкті GET /experiments повертає список експериментів, GET /experiments/{id} — один експеримент, а GET /experiments/statistics — статистику.

7. Для чого використовується POST?

POST використовується для створення нового ресурсу або запуску обробки, причому дані передаються в тілі запиту, зазвичай у JSON. Метод не ідемпотентний: повторний запит створить ще один запис. У проєкті POST /experiments і POST /researchers створюють нові об'єкти й повертають 201 Created.

8. Для чого використовується PUT?

PUT використовується для повної заміни ресурсу за відомим URL: клієнт надсилає повне представлення об'єкта, і поля, яких немає в тілі, не зберігаються, а замінюються. PUT ідемпотентний, бо повторний однаковий запит дає той самий стан. У проєкті для оновлення експерименту замість PUT використано PATCH.

9. Для чого використовується PATCH?

PATCH використовується для часткового оновлення ресурсу: клієнт надсилає лише ті поля, які треба змінити. У проєкті PATCH /experiments/{id} приймає Pydantic-схему, у якій усі поля необов'язкові, і застосовує лише передані поля через model_dump(exclude_unset=True), тому непередані поля не перезаписуються значенням None.

10. Для чого використовується DELETE?

DELETE використовується для видалення ресурсу за його ідентифікатором; метод ідемпотентний, бо після першого успішного видалення стан сервера вже не змінюється. У проєкті DELETE /experiments/{id} видаляє експеримент і повертає 204 No Content, а для неіснуючого id — 404 Not Found.

11. Що означає status code 200?

Код 200 OK означає, що запит успішно оброблено, а тіло відповіді містить результат, наприклад запитаний ресурс чи список. Це стандартний код FastAPI для звичайних endpoints, зокрема GET і PATCH; у проєкті GET /experiments повертає 200 зі списком експериментів у JSON.

12. Що означає 201?

Код 201 Created означає, що запит успішно виконано і на сервері створено новий ресурс; у відповіді зазвичай повертають створений об'єкт, іноді із заголовком Location. У проєкті POST /experiments і POST /researchers оголошено з status_code=201, бо за замовчуванням FastAPI повертає 200.

13. Що означає 204?

Код 204 No Content означає, що запит виконано успішно, але тіла відповіді немає. Його типово повертають на DELETE або на оновлення, після якого дані повертати не потрібно; у проєкті DELETE /experiments/{id} повертає 204 після видалення експерименту.

14. Що означає 404?

Код 404 Not Found означає, що ресурсу за вказаним URL не існує, наприклад експерименту з таким id немає в БД. У FastAPI його повертають через raise HTTPException(status_code=404, detail=...), а для невідомого шляху фреймворк повертає 404 автоматично.

15. Що означає 422?

Код 422 Unprocessable Entity (Validation Error) означає, що запит синтаксично коректний, але дані не пройшли валідацію: бракує обов'язкового поля, тип неправильний або значення поза допустимими межами. FastAPI повертає його автоматично, коли Pydantic відхиляє тіло, path- чи query-параметри, і в полі detail перелічує помилки з місцем кожної.

16. Що таке JSON?

JSON (JavaScript Object Notation) — текстовий, незалежний від мови програмування формат обміну структурованими даними, що підтримує об'єкти {}, масиви [], рядки, числа, true/false і null. Він є стандартним форматом тіла запитів і відповідей REST API; FastAPI автоматично серіалізує результати в JSON, а в Python з ним працює модуль json.

17. Що таке REST?

REST (Representational State Transfer) — архітектурний стиль побудови веб-API, у якому система описується як набір ресурсів з унікальними URL, а дії над ними виконуються стандартними HTTP-методами. Основні принципи: клієнт-серверна взаємодія, відсутність стану між запитами (stateless), єдиний інтерфейс, змістовні коди стану й представлення ресурсів, зазвичай у JSON.

18. Що таке resource у REST?

Resource у REST — сутність предметної області, з якою працює API і яка має власний URI, наприклад колекція /experiments або окремий експеримент /experiments/{id}. Клієнт отримує не сам об'єкт, а його представлення (зазвичай JSON); у проєкті ресурсами є експерименти й дослідники, а вкладений шлях /researchers/{id}/experiments повертає експерименти конкретного дослідника.

19. Що таке endpoint?

Endpoint — конкретна точка доступу API, тобто поєднання HTTP-методу та шляху, яку обробляє окрема функція. У FastAPI endpoint оголошується декоратором на зразок @app.get(...) над функцією-обробником; у проєкті GET /experiments і POST /experiments — два різні endpoints для одного шляху.

20. Що таке path parameter?

Path parameter — змінна частина шляху URL, яка зазвичай ідентифікує конкретний ресурс, наприклад {id} у /experiments/{id}. FastAPI витягає її значення, перетворює до типу з анотації параметра функції (наприклад, int) і повертає 422, якщо перетворення неможливе; запит GET /experiments/5 передасть у функцію значення 5.

21. Що таке query parameter?

Query parameter — параметр, що передається після знака ? в URL у форматі ключ=значення, розділених &, і зазвичай використовується для фільтрації, сортування чи пагінації, наприклад ?limit=10&offset=20. У FastAPI ним стає параметр функції простого типу, якого немає в шляху; значення за замовчуванням робить його необов'язковим.

22. Що таке request body?

Request body — дані, які клієнт передає в тілі HTTP-запиту, зазвичай у JSON із заголовком Content-Type: application/json; його використовують POST, PUT і PATCH. У FastAPI тіло описується параметром типу Pydantic-моделі, яка автоматично розбирає й валідує JSON; у проєкті тілом POST /experiments є дані нового експерименту.

23. Що таке FastAPI?

FastAPI — сучасний асинхронний Python-фреймворк для створення веб-API, побудований на Starlette (ASGI) і Pydantic. Він використовує анотації типів для валідації й серіалізації даних, підтримує обробники async def і dependency injection та автоматично генерує документацію OpenAPI; у проєкті застосунок доступний як experiment_registry.api:app.

24. Для чого використовується Uvicorn?

Uvicorn — ASGI-сервер, який приймає HTTP-з'єднання, передає запити ASGI-застосунку (FastAPI) і повертає клієнтам відповіді, працюючи на основі event loop asyncio. Проєкт запускається командою uvicorn experiment_registry.api:app --reload, де --reload автоматично перезапускає сервер при зміні коду під час розробки.

25. Для чого потрібний Swagger UI?

Swagger UI — інтерактивна веб-документація API, яку FastAPI автоматично генерує зі схеми OpenAPI і відкриває за адресою /docs (альтернативний вигляд — ReDoc на /redoc). Вона дозволяє переглядати endpoints, параметри й Pydantic-схеми, а також надсилати реальні запити прямо з браузера та аналізувати відповіді.

26. Що таке Pydantic?

Pydantic — бібліотека для валідації, парсингу й серіалізації даних на основі анотацій типів: схеми успадковуються від BaseModel, а обмеження задаються через Field (min_length, ge, le тощо). У проєкті використовується Pydantic v2, де основні методи — model_validate() для створення моделі з даних і model_dump() для перетворення моделі в словник.

27. Що таке validation?

Validation — перевірка того, що вхідні дані відповідають очікуваній структурі, типам і обмеженням, ще до того як вони потраплять у бізнес-логіку чи БД. У FastAPI її автоматично виконує Pydantic: некоректні дані, наприклад рядок замість числа чи відсутнє обов'язкове поле, відхиляються з кодом 422 і описом помилок.

28. Що таке response model?

Response model — Pydantic-схема, яка описує структуру відповіді endpoint-а й задається параметром response_model або анотацією типу повернення. FastAPI перетворює повернений об'єкт у цю схему (для ORM-об'єктів потрібне model_config = {"from_attributes": True}), відкидає зайві поля, валідує дані й документує відповідь в OpenAPI.

29. Що таке synchronous programming?

Synchronous programming — модель виконання, у якій операції виконуються послідовно, і кожна наступна починається лише після завершення попередньої. Під час блокувального вводу-виводу (мережевий запит, читання файлу, звернення до БД) потік простоює, тому кілька HTTP-запитів займають час, що дорівнює сумі їхніх тривалостей.

30. Що таке asynchronous programming?

Asynchronous programming — модель, у якій програма під час очікування вводу-виводу не блокується, а передає керування іншим задачам, тож багато операцій очікують одночасно в одному потоці. У Python її реалізують async/await і asyncio; вона ефективна для I/O-bound задач (HTTP, БД, мережа), але не прискорює CPU-bound обчислення.

31. Для чого використовується async?

Ключове слово async у конструкції async def оголошує корутинну функцію: її виклик не виконує тіло одразу, а повертає об'єкт coroutine, який треба запустити через await, Task або asyncio.run(). Також async with і async for працюють з асинхронними контекстними менеджерами та ітераторами, наприклад async with httpx.AsyncClient() as client.

32. Для чого використовується await?

await призупиняє поточну корутину до завершення awaitable-об'єкта (coroutine, Task, Future) і повертає керування event loop, який тим часом виконує інші задачі; після завершення await повертає результат або піднімає виняток. Його можна використовувати лише всередині async def, наприклад response = await client.get(url).

33. Що таке coroutine?

Coroutine (корутина) — об'єкт, який повертає виклик функції, оголошеної через async def; її виконання можна призупиняти в точках await і згодом відновлювати. Сама корутина нічого не робить, доки її не очікують через await, не загорнуть у Task або не запустять через asyncio.run().

34. Що таке event loop?

Event loop (цикл подій) — центральний механізм asyncio, який в одному потоці планує й виконує корутини та задачі, відстежує готовність операцій вводу-виводу і таймерів та відновлює задачі, коли очікувана подія настала. Його створює й запускає asyncio.run(), а для FastAPI-застосунку цикл подій забезпечує Uvicorn.

35. Що таке Task?

Task (asyncio.Task) — обгортка над корутиною, яка планує її виконання в event loop, щоб вона працювала конкурентно з іншими задачами. Task є awaitable-об'єктом: на нього можна чекати через await, отримати результат або виняток, перевірити стан методом done() чи скасувати методом cancel().

36. Для чого використовується asyncio.create_task()?

asyncio.create_task() загортає корутину в Task і одразу планує її виконання в поточному event loop, не чекаючи завершення, тому кілька задач починають

виконуватися конкурентно. На створені задачі слід зберігати посилання й потім очікувати їх; у проєкті для кожного дослідника створюється окрема задача агрегації статистики.

37. Для чого використовується asyncio.gather()?

asyncio.gather() конкурентно виконує кілька awaitable-об'єктів, чекає завершення всіх і повертає список результатів у тому ж порядку, у якому їх передали. За замовчуванням перший виняток передається тому, хто викликав gather, а з return_exceptions=True винятки повертаються як елементи списку; у проєкті gather збирає статистику кількох дослідників із задач, створених через create_task.

38. Що таке timeout?

Timeout — максимальний час очікування операції, після якого вона переривається з помилкою, щоб програма не зависала через повільний чи недоступний сервер. У проєкті використано asyncio.wait_for(), яка скасовує корутину й піднімає TimeoutError, і параметр timeout у httpx, що обмежує з'єднання, читання й запис та піднімає httpx.TimeoutException.

39. Для чого потрібний retry?

Retry — повторне виконання операції після тимчасової помилки, наприклад мережевого збою, таймауту чи відповіді 5xx, з обмеженою кількістю спроб. Він підвищує стійкість до короткочасних збоїв; повторювати варто лише тимчасові помилки, а не 404 чи 422, і після останньої невдалої спроби виняток слід передати далі. У проєкті retry поєднано з експоненційним backoff.

40. Що таке backoff?

Backoff — стратегія збільшення паузи між повторними спробами, щоб не перевантажувати сервер, який уже має проблеми, і дати йому час відновитися. У проєкті використовується exponential backoff: затримка подвоюється з кожною спробою (наприклад, 0.5, 1, 2 секунди), а пауза робиться через await asyncio.sleep(), щоб не блокувати event loop; часто додають ще й випадковий jitter.

41. Для чого використовується Semaphore?

asyncio.Semaphore обмежує кількість корутин, які одночасно виконують певну ділянку коду: він має лічильник дозволів, і конструкція async with semaphore чекає, доки звільниться дозвіл. Наприклад, Semaphore(5) пропускає не більше 5 одночасних операцій; у проєкті він обмежує кількість паралельних запитів статистики дослідників.

42. Що таке HTTPX?

HTTPX — HTTP-клієнт для Python із синхронним (httpx.Client) та асинхронним (httpx.AsyncClient) API, схожим на requests, з підтримкою таймаутів, пулу з'єднань і, опційно, HTTP/2. Через ASGITransport він може надсилати запити безпосередньо ASGI-застосунку без мережі; у проєкті HTTPX використовується для асинхронних запитів статистики та в тестах API.

43. Чим httpx.AsyncClient відрізняється від синхронного клієнта?

Синхронний httpx.Client виконує запити блокувально: виклик client.get() зупиняє потік до отримання відповіді. Методи httpx.AsyncClient є корутинами, які викликаються через await і під час очікування мережі віддають керування event loop, тому багато запитів виконуються конкурентно в одному потоці; відкривати й закривати його слід через async with.

44. Чому AsyncClient бажано повторно використовувати?

AsyncClient тримає пул з'єднань і повторно використовує TCP- та TLS-з'єднання з тим самим хостом, що усуває витрати на встановлення нового з'єднання для кожного запиту. Крім того, спільні налаштування (base_url, timeout, заголовки) задаються один раз; тому в проєкті один httpx.AsyncClient відкривається через async with і використовується всіма задачами.

45. Навіщо обмежувати паралелізм?

Обмеження паралелізму захищає від перевантаження зовнішнього сервера чи БД, перевищення лімітів запитів (rate limit, відповідь 429), вичерпання пулу з'єднань, сокетів і пам'яті. Воно робить навантаження передбачуваним і зменшує кількість таймаутів; у проєкті цю роль виконує asyncio.Semaphore.

46. Чому не слід запускати тисячі tasks одночасно?

Тисячі одночасних tasks означають тисячі одночасних з'єднань і запитів: сервер може відповідати 429 чи 503 або перестати відповідати, клієнт вичерпає пул з'єднань і файлові дескриптори, зросте споживання пам'яті, а запити почнуть масово завершуватися таймаутами. Тому кількість одночасно активних операцій обмежують через Semaphore або обробляють задачі пакетами.

47. Як REST API взаємодіє з Repository Layer?

Обробник endpoint-а отримує валідовані Pydantic-схемою дані, відкриває сесію БД (наприклад, через Depends), викликає методи репозиторію або сервісу, а повернений ORM-об'єкт перетворюється на response model. Доменні ситуації перетворюються на HTTP-коди, наприклад відсутній запис на 404; так API-шар не містить SQL, а ResearcherRepository та ExperimentRepository нічого не знають про HTTP.

48. Як FastAPI працює з Pydantic?

FastAPI аналізує анотації параметрів обробника: параметр типу Pydantic-моделі вважається тілом запиту, яке автоматично парситься з JSON і валідується, а при помилці повертається 422. Результат функції проходить через response model для фільтрації та серіалізації в JSON, а з тих самих схем генерується OpenAPI-документація; у проєкті так працюють Pydantic v2 схеми експериментів і дослідників.

49. Як протестувати REST API?

REST API тестують, надсилаючи HTTP-запити до застосунку й перевіряючи код стану, JSON-тіло та зміни в БД, зокрема негативні сценарії (404, 422). У проєкті синхронні тести використовують fastapi TestClient, а асинхронні — httpx.AsyncClient з ASGITransport(app=app), що викликає застосунок напряму без запуску сервера; вручну запити можна перевірити через Swagger UI на /docs.

50. Чим API integration test відрізняється від unit test?

Unit test перевіряє ізольовану функцію чи клас, наприклад обчислення статистики, без HTTP і реальної БД, підміняючи залежності. API integration test надсилає HTTP-запит до застосунку через TestClient або ASGITransport (без реальної мережі) і перевіряє весь ланцюжок: маршрутизацію, валідацію Pydantic, роботу репозиторію з тестовою БД, серіалізацію відповіді й коди стану, тому він повільніший, але ближчий до реального використання.

6. Висновок

У лабораторній роботі проєкт experiment-registry отримав асинхронний REST API на FastAPI поверх persistence layer лабораторної №7. Реалізовано всі endpoints варіанта 15: CRUD експериментів і дослідників, пошук за дослідником, зміну результату, видалення, average/minimum/maximum, фільтрацію за діапазоном і сортування за датою. Pydantic-схеми перевіряють JSON, а коректні коди 200/201/204/404/409/422 повертаються як для валідації, так і для доменних помилок.

Асинхронна частина поєднує `asyncio.Task`, `gather`, `Semaphore`, `wait_for` і HTTPX-клієнт з retry та exponential backoff: статистика для кількох дослідників отримується паралельно, а збій одного запиту не впливає на інші. 37 API-тестів (TestClient, AsyncClient, MockTransport) і загалом 221 тестів проєкту проходять; документація API доступна в Swagger UI.